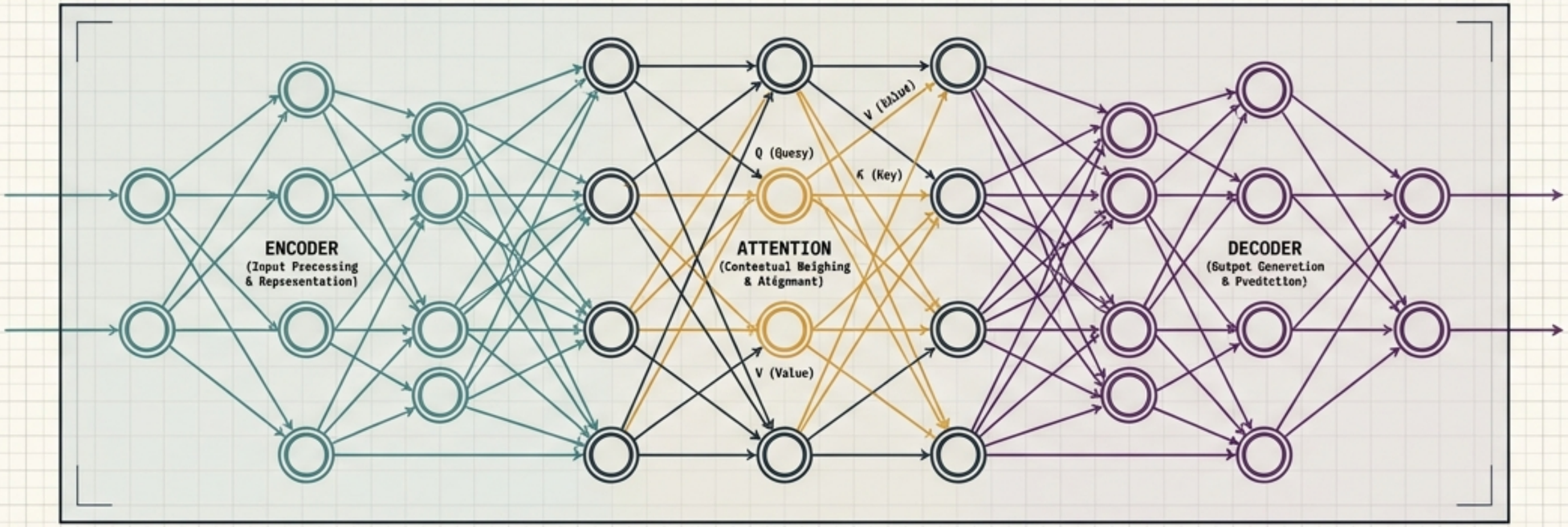
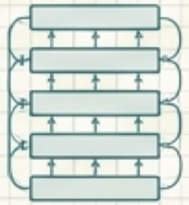
The Transformer Schematic

Demystifying the Architecture Behind BERT, GPT, and Modern LLMs

An architectural breakdown of Self-Attention, Encoders, Decoders, and the evolutionary branches of Large Language Models.



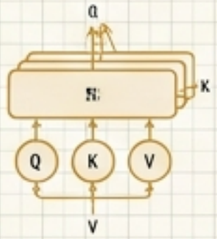
ENCODER: BERT & Autoencoding



Bidirectional Context
Input: `[CLS] My dog is cute [SEP] He plays fetch. [SEP]` -> Tokens
Process: `Masked LM (MLM)`, `Next Sentence Prediction (NSP)`
Output: `Contextualized Embeddings`
Evolution: BERT, RoBERTa, DistilBERT.

```
class BERTEncoder(nn.Module):
  def forward(self, input_ids):
    # Process...
    return embeddings
```

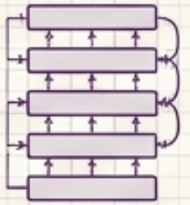
ATTENTION: The Core Mechanism



Self-Attention (Multi-Head)
Formula: `Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V`
Key Concepts: `Query`, `Key`, `Value`, `Scaled Dot-Product`, `Multi-Head`
Importance: Captures long-range dependencies.

```
def scaled_dot_product_attention(q, k, v, mask):
  scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
  # Apply mask...
  attn = torch.softmax(scores, dim=-1)
  output = torch.matmul(attn, v)
  return output
```

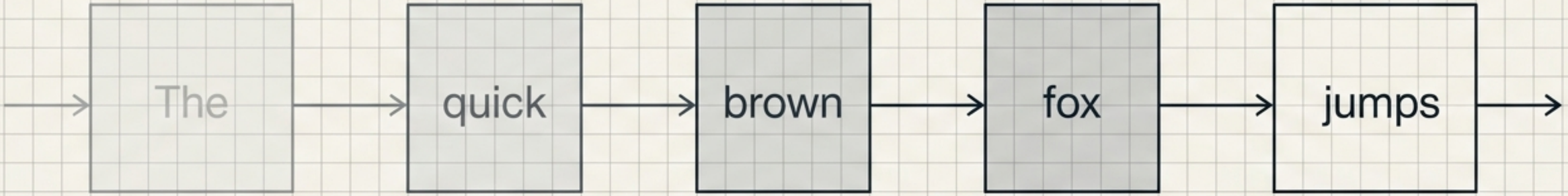
DECODER: GPT & Autoregressive



Unidirectional Context
Input: `[START] The quick brown fox` -> Tokens
Process: `Causal Language Modeling (CLM)`, `Next Token Prediction`
Output: `Generated Text / Probabilities`
Evolution: GPT-2, GPT-3, ChatGPT, LLaMA.

```
class GPTDecoder(nn.Module):
  def forward(self, input_ids, past_key_values):
    # Process...
    # Output...
    return logits, past_key_values
```

The sequential processing bottleneck of early language models



The Legacy Engine: RNNs, LSTMs, and GRUs processed text sequentially (word-by-word).

⚠ [ERROR 01]

Slow Sequential Processing: Cannot process multiple words at once.

⚠ [ERROR 02]

Amnesia: Forgets long-term context as sequences grow.

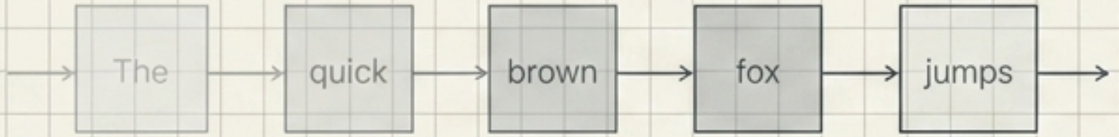
⚠ [ERROR 03]

Vanishing Gradients: Mathematical instability during training.

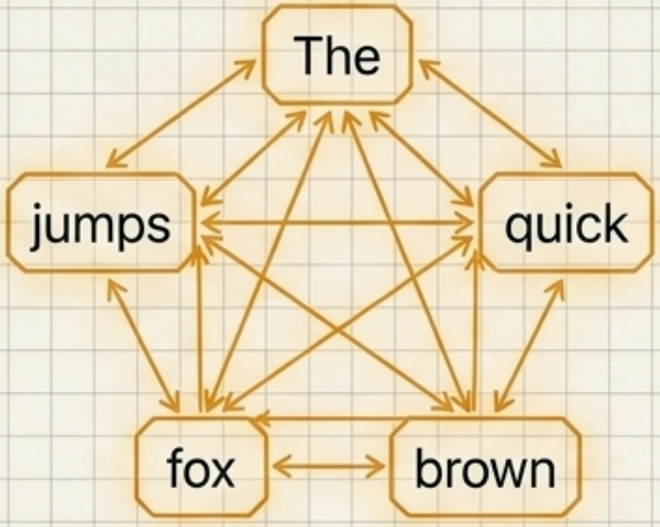
⚠ [ERROR 04]

Hard Parallelization: Incompatible with modern GPU hardware scaling.

The 2017 breakthrough: Replacing sequence with parallel context



Legacy Sequence: Word-by-word



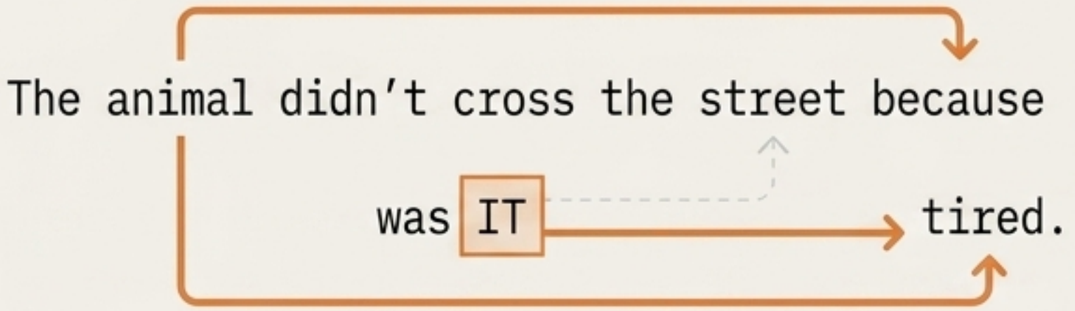
The Catalyst: The `Attention Is All You Need` paper introduced a radical solution—drop the sequential processing entirely.

The Core Shift: Instead of reading words one-by-one, the Transformer looks at ALL words together using `Attention`.

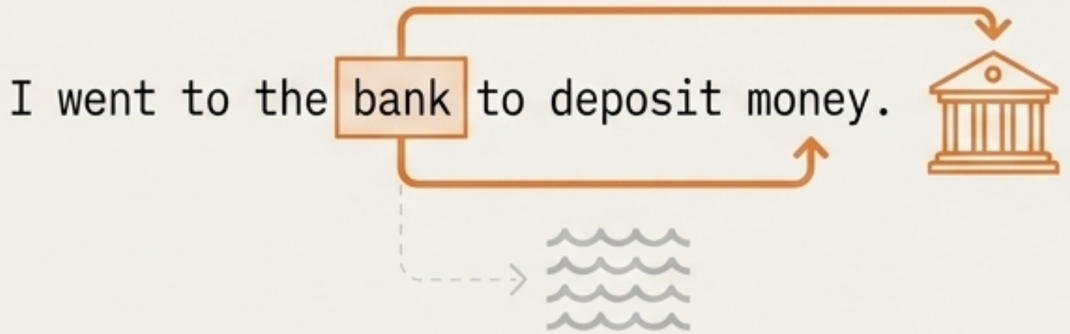
- ☑ `Self-Attention`: Weighs the importance of all surrounding context.
- ☑ `Parallel Processing`: Massively accelerates training on GPUs.
- ☑ Long-Range Dependency: Connects words regardless of their distance in the sentence.

Self-Attention connects distant concepts and disambiguates meaning

PANEL 1: Disambiguating Reference



PANEL 2: Resolving Homonyms with Context



The Intuition:
Imagine a classroom where every student **listens** to every other student, but **pays more attention** to the important ones.

The Core Shift: Self-attention captures relationships between distant words, allowing the model to dynamically update the meaning of a word based on its neighbors.

The Input Pipeline transforms words into spatial coordinates

1. Tokenization

Text is split into sub-word units or tokens.

Give an example.

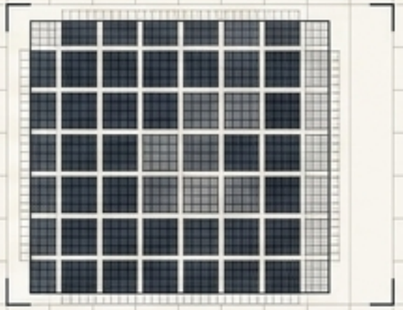
[Give, an, example, .]

2. Token IDs

[345, 621, 2367, 13]

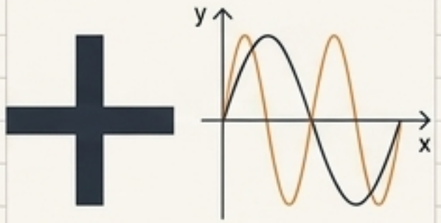
3. Embeddings

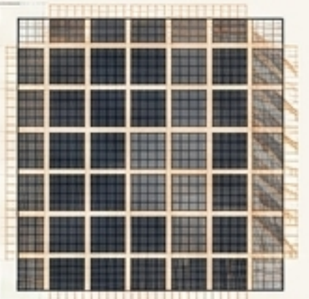
Words are converted into high-dimensional vectors representing their semantic meaning.



4. Positional Encoding

Because Transformers process everything at once, they lack sequence awareness. Positional data is injected to tell the model where the word sits in the sentence. Without it, "Dog bites man" and "Man bites dog" look identical.

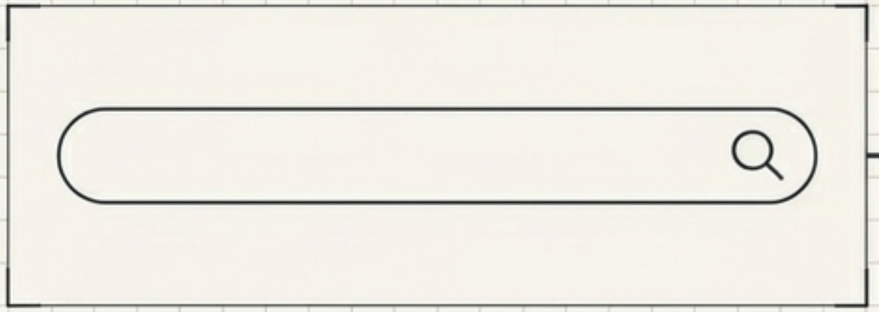




The Q, K, V mechanism: A dynamic retrieval system

Every word in the sequence generates three distinct mathematical vectors:

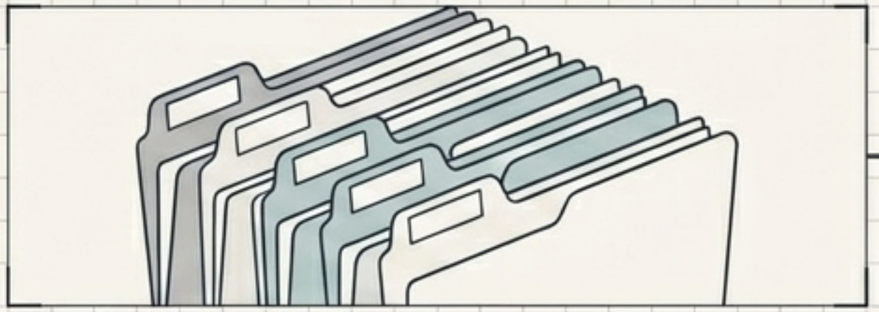
Q



Query (Q)

What am I looking for?
(The current word searching for context).

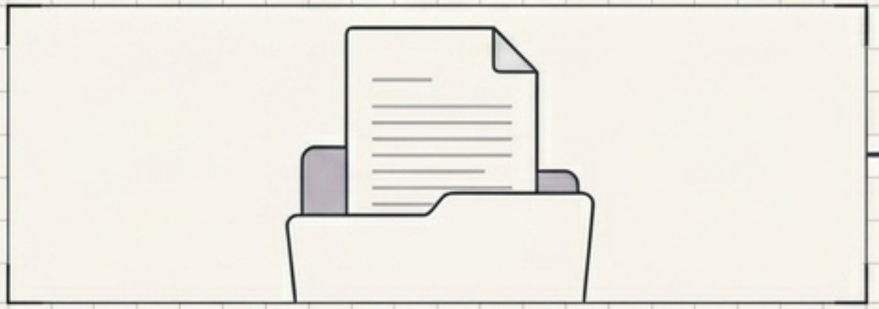
K



Key (K)

What information do I contain?
(The context offered by other words).

V



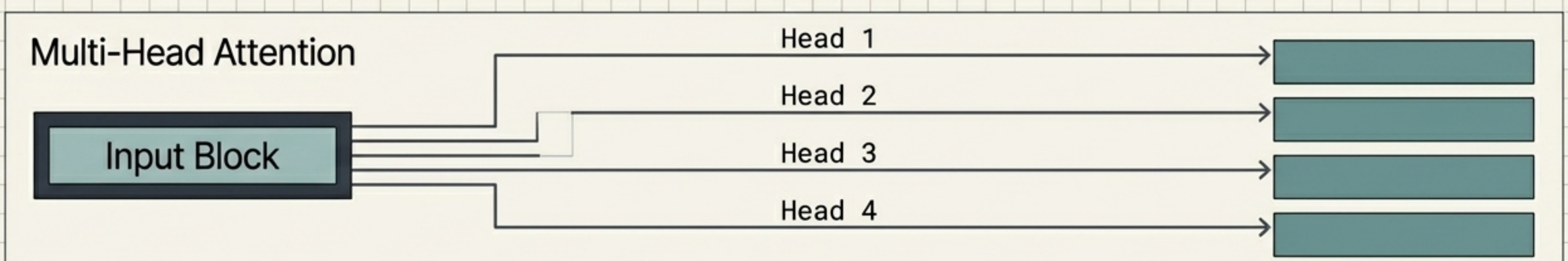
Value (V)

What is my actual meaning?
(The information passed forward if a match occurs).

Calculating Attention and splitting across multiple heads

$$\textbf{Attention}(Q,K,V) = \textbf{softmax}(QK^T/\sqrt{d_k})V$$

- Measures the similarity between words.
- A scaling factor that prevents extremely large dot products, ensuring mathematical stability.
- Converts raw similarity scores into percentages (probabilities 0.0 to 1.0).
- Multiplies the probabilities by the actual values to extract important info.



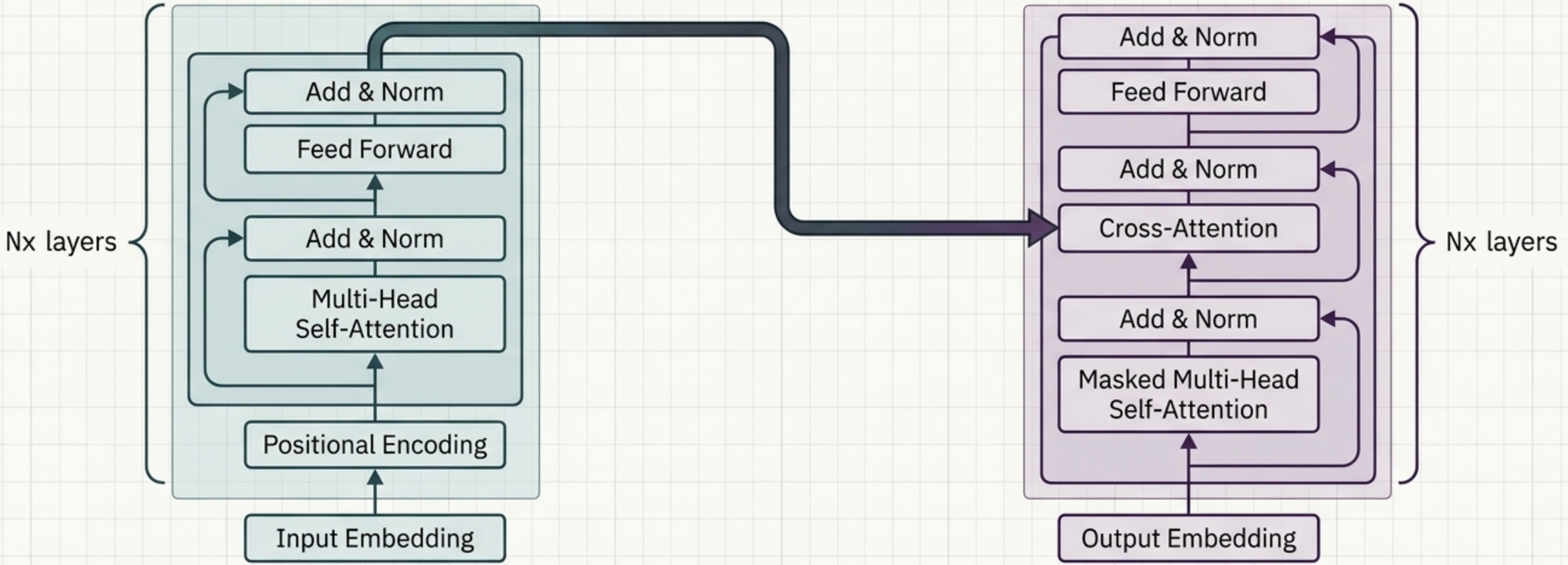
Multi-Head Attention: Instead of one attention mechanism, multiple heads run in parallel. Each head learns a different relationship (e.g., Head 1: Grammar, Head 2: Emotion, Head 3: Syntax, Head 4: Semantic Meaning).

The original architectural blueprint: Encoder + Decoder

The original Transformer was designed for sequence-to-sequence mapping (like Translation).

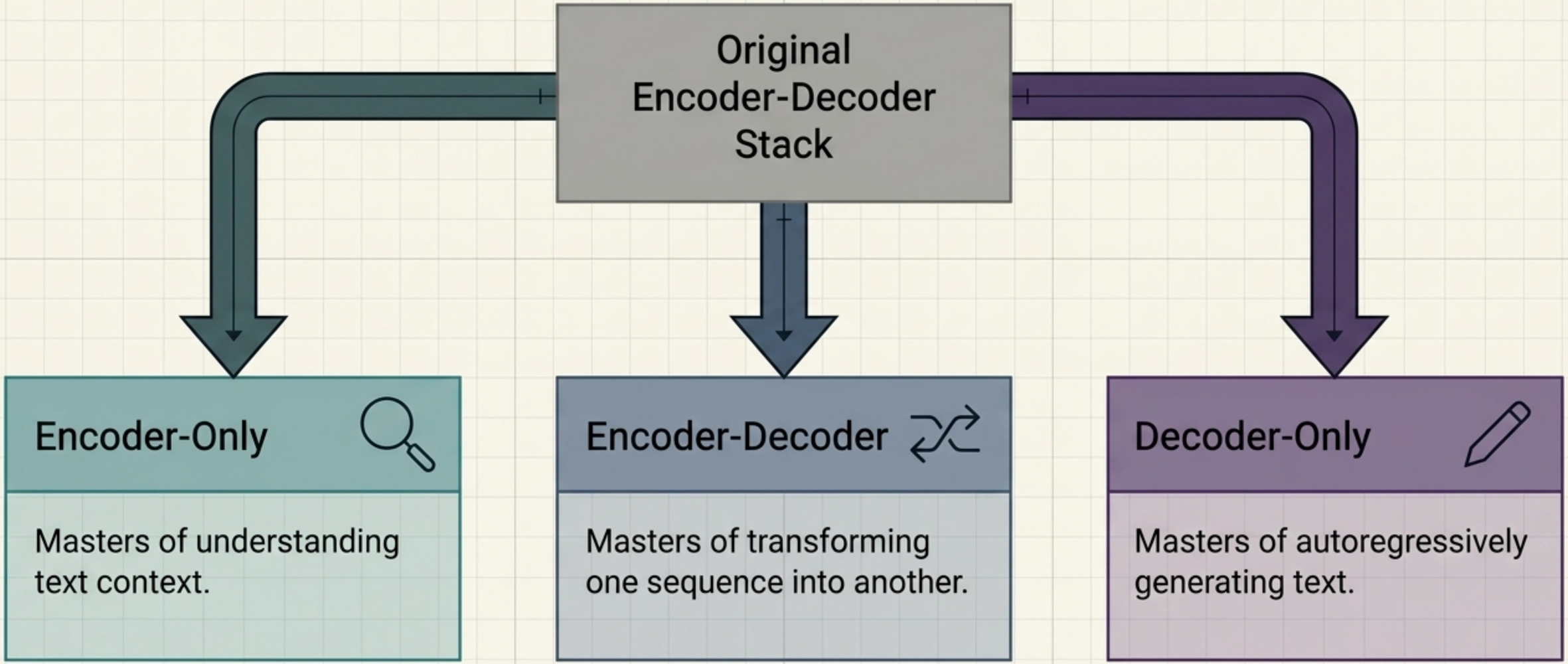
Encoder stack
Reads the source input simultaneously to build a deep, contextualized understanding.

Decoder stack
Takes the Encoder's context and generates the output sequence step-by-step.



The Evolutionary Branching of Large Language Models

Modern LLMs are simply different configurations of the original architecture.



BERT: Deep understanding through bidirectional context

Traditional Models

BERT (Bidirectional Encoder Representations from Transformers)

Category: Encoder-Only Transformer.

The Innovation: Older models read text strictly Left -> Right. BERT reads the entire sequence simultaneously in BOTH directions (Left <-> Right).

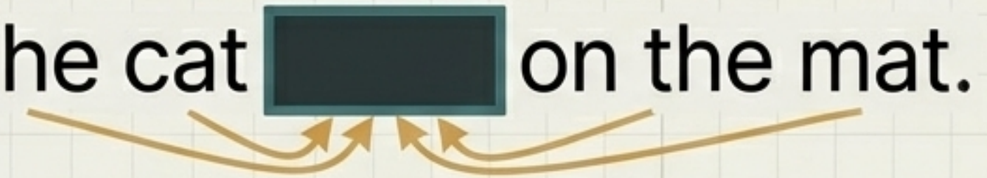
Strengths: Deep bidirectional understanding, state-of-the-art NLP classification, powerful context learning.

Use Cases: Search engines, document classification, spam detection, Named Entity Recognition (NER).

Training Encoders: MLM, NSP, and the RoBERTa upgrade

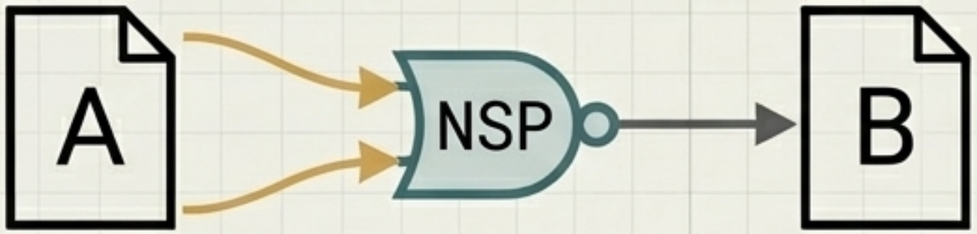
How BERT Learns

MLM (Masked Language Modeling)



Hides a word with a [MASK] token. Forces the model to understand context to predict it.

NSP (Next Sentence Prediction)

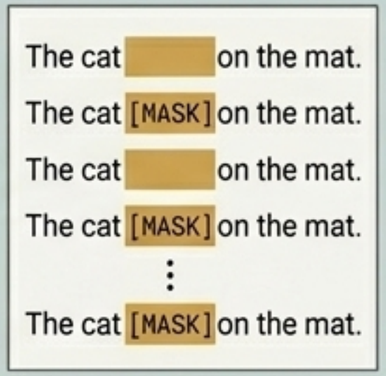


Predicts if Sentence B logically follows Sentence A to learn dialogue flow.

RoBERTa (Robustly Optimized BERT Approach) Patch Notes

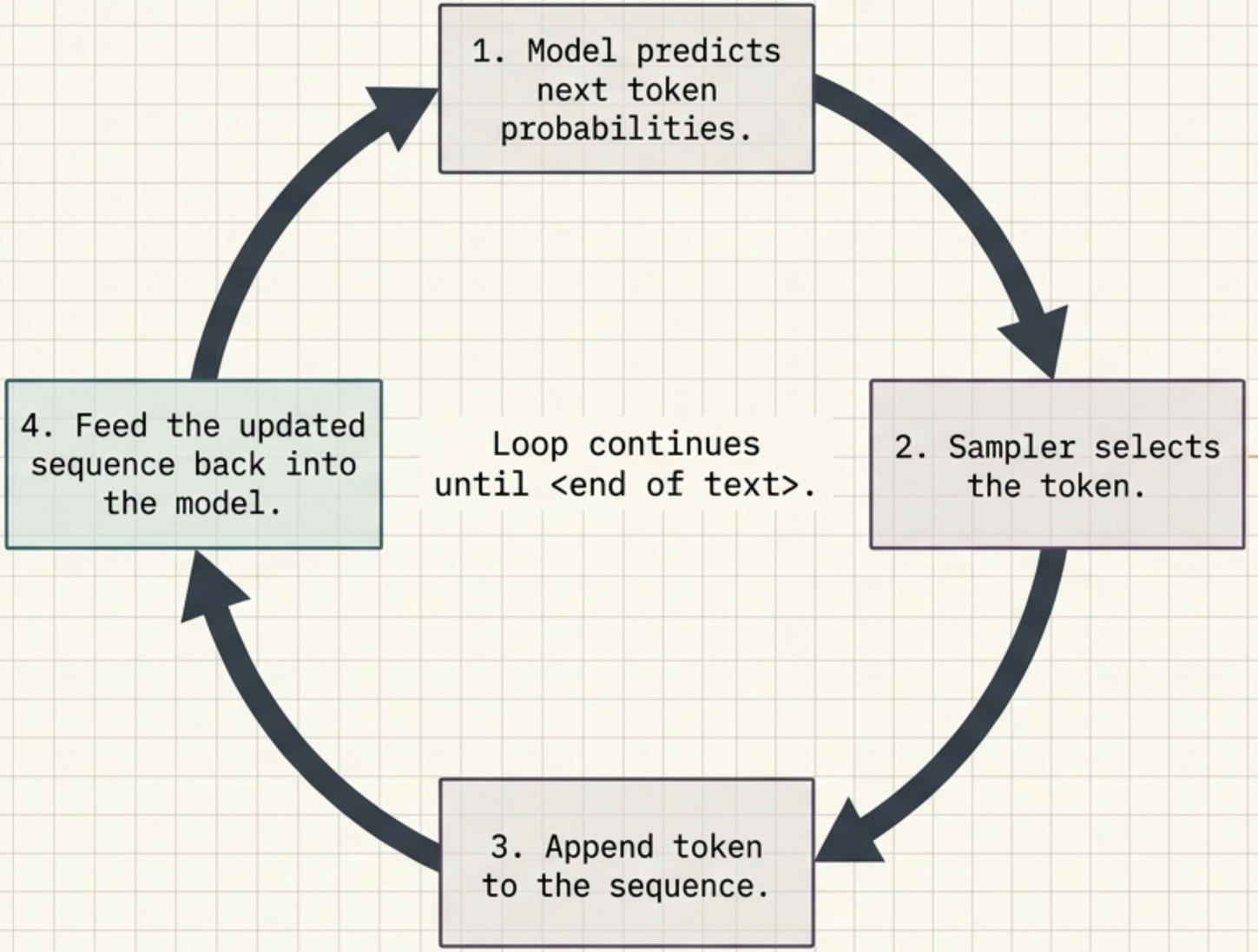
[-] **Removed NSP:** Researchers found it mathematically unnecessary.

[+] **Dynamic Masking:** [MASK] tokens change positions every epoch, forcing better learning compared to BERT's static masks.



[+] **Scaled Up:** Trained on massively larger datasets with larger batch sizes for higher accuracy.

GPT: Autoregressive text generation using Decoders



Category:

Decoder-Only Transformer.

Architecture:

GPT (Generative Pre-trained Transformer).

The Core Engine:

Predicts the highly probable next token based on all previous tokens in the sequence.

Autoregressive Generation:

Generates words sequentially. The output becomes the new input.

Limitations:

Computationally expensive, prone to hallucinations (confident factual errors), cannot look ahead.

Causal Masking prevents Decoders from looking into the future

	T1	T2	T3	T4	T5
T1	✓	✗	✗	✗	✗
T2	✓	✓	✗	✗	✗
T3	✓	✓	✓	✗	✗
T4	✓	✓	✓	✓	✗
T5	✓	✓	✓	✓	✓

The Cheating Problem:
Because Transformers process all text simultaneously, a Decoder trying to learn how to predict the next word could accidentally ‘see’ the answer later in the training sentence.

The Masking Solution:
Causal masking applies a mathematical block (**red X**) that prevents the current position from attending to any future positions.

Result:
T3 can look at T1, T2, and T3. But T3 is mathematically blind to T4 and T5.

The Large Language Model Taxonomy Matrix

Variant Type	Main Task	Core Mechanism	Input -> Output	Famous Models
Encoder-Only	Understanding	Bidirectional Self-Attention	`Input -> Representations`	BERT, RoBERTa
Decoder-Only	Generation	Causal (Masked) Self-Attention	`Input -> Autoregressive Output`	GPT-4, LLaMA, Mistral
Encoder-Decoder	Sequence-to-Sequence	Bidirectional + Cross-Attention	`Input -> Different Output`	T5, BART

Quick-Recall Cheat Sheet & Interview Prep

Q: Why are Transformers better than RNNs?

A: Parallel processing, massive GPU scaling, and superior long-range dependency tracking.

Q: What exact problem does Attention solve?

A: It captures deep contextual relationships between distant words, dynamically shifting meanings based neighbors.

Q: Why is Positional Encoding necessary?

A: Because Transformers ingest all data at once, they inherently lack sequence awareness.

Q: Why is a scaling factor ($\sqrt{d_k}$) used?

A: It stabilizes training by preventing extremely large dot products during softmax calculation.

Q: Why do Decoders use Masking?

A: To prevent tokens from "cheating" by viewing future token visibility during generation.

The One-Line Memory Trick

Encoder-only = **Understand.**
Decoder-only = **Generate.**
Encoder-Decoder = **Translate.**